

Coding & Solution

Date	02 July 2026
Team ID	xxxxxx
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Harshithalingineni/A-Comprehensive-Measure-of-Well-Being
Programming Language(s)	Python 3.11
Framework(s) / Libraries Used	Flask (Web Framework), Scikit-learn (ML), Pandas & NumPy (Data Processing), Matplotlib & Seaborn (Visualization)
Key Features Implemented	1. Data preprocessing and handling missing values. 2. Exploratory Data Analysis (EDA) with correlation heatmaps and pairplots. 3. Machine Learning model training (Random Forest Classifier) to predict HDI tiers (Very High, High, Medium, Low). 4. Interactive Flask web application for user input and real-time HDI prediction. 5. Result visualization and tier classification display.
Pending / Incomplete Features	1. Cloud deployment (e.g., AWS/Heroku). 2. Batch prediction via CSV upload. 3. User authentication for saving prediction history.
Setup / Run Instructions	1. Clone the repository: <code>git clone <repo-url></code> 2. Navigate to the project directory: <code>cd hdi-prediction</code> 3. Install dependencies: <code>pip install -r requirements.txt</code> 4. Run the Flask application: <code>python app.py</code> 5. Open the browser and go to <code>http://127.0.0.1:5000/</code> to access the UI.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

- **Code Standards:** The codebase strictly follows **PEP-8** guidelines for Python, ensuring consistent indentation, spacing, and naming conventions (e.g., snake_case for variables/functions).
- **Modularity:** The project is divided into logical modules such as data_processing.py, model_training.py, and app.py (Flask routes) to ensure high reusability and maintainability.
- **Error Handling:** Form inputs in the Flask application are validated to ensure users enter numerical values within realistic ranges for life expectancy, schooling years, and GNI. Invalid inputs trigger user-friendly error messages.
- **Team Collaboration:** The team utilized Git for version control, with each member working on separate branches for features like EDA, model tuning, and frontend integration before merging into the main branch.
- **Future Scope:** The current model can be further enhanced by integrating deep learning techniques or adding a time-series forecasting feature to predict future HDI trends based on historical data.